

Solo Trading Operating System: Master Plan

1. Executive decision

Build an **institutional-quality, multi-asset trading operating system for advanced independent traders and small professional teams.**

The platform will unify:

Market data → research → strategy development → backtesting → paper trading → live execution → portfolio risk → monitoring → journaling → reporting

The primary differentiator will be:

A strategy behaves identically in research, simulation, paper trading and live trading, with deterministic replay, enforced risk controls and a complete audit trail.

This is achievable for one exceptional developer.

Building a Jane Street-equivalent system is not achievable solo. Jane Street-level low-latency trading depends on specialized teams, direct exchange connectivity, proprietary infrastructure and physical proximity to exchanges. Nasdaq's own co-location service advertises microsecond-scale latency improvements and sub-50-microsecond connectivity characteristics, which ordinary cloud and broker APIs cannot match.

Therefore, this product will **not** compete in high-frequency trading. It will compete in correctness, integration, safety, usability and research-to-production velocity.

2. Corrected constraints

Original ambition	Corrected constraint
Support every market immediately	Architect for every market; implement one broker and one asset class first
Perform every trading action	Support the complete professional workflow, but release modules sequentially
Match Jane Street	Match institutional engineering principles, not its latency, capital or market breadth
Build everything from scratch	Build the proprietary trading domain from scratch; use proven databases, cryptography, UI frameworks and cloud infrastructure

Original ambition	Corrected constraint
Be operated by one person	Begin as a modular monolith with managed infrastructure and limited operational complexity
Guarantee \$1 million value	Build toward measurable revenue, retention, IP, security and customer trust that could justify such a valuation
Fully automated from day one	Replay first, paper trading second, limited-capital live trading third
Global launch	One commercial jurisdiction first; add jurisdictions only after legal review
Produce profitable strategies	Provide infrastructure, analytics and execution—not profit guarantees or investment promises

Hard non-goals

The first two years must exclude:

- High-frequency or latency-arbitrage trading.
- Exchange membership or direct market access.
- Custody of client assets.
- Managing pooled customer money.
- Personalized investment advice.
- Copy trading or a public strategy marketplace.
- Ten broker integrations.
- Mobile applications.
- Social features.
- Custom database engines.
- Custom cryptography.
- Kubernetes before genuine operational need.
- AI-generated autonomous strategies trading without human-defined limits.

These exclusions protect the project from becoming permanently unfinished.

3. Product definition

Working category

Professional Multi-Asset Trading OS

Initial ideal customer

The best initial customer profile is:

- Independent systematic traders.
- Technically sophisticated discretionary traders.
- Small proprietary trading teams.
- Small funds with one to five traders.
- Developers currently combining notebooks, charting software, spreadsheets, broker terminals and custom scripts.

Do not target beginning retail traders first. Beginners require education, support and simplified workflows, while professional users are more likely to pay for reliability, reproducibility and risk controls.

Product promise

Develop, validate, deploy and supervise systematic or discretionary trading workflows from one auditable platform.

The commercial wedge

The wedge is not “more indicators” or “AI predictions.”

The wedge is:

1. **Research/live parity:** the same strategy API and event models everywhere.
2. **Deterministic replay:** reproduce exactly what the system knew and did.
3. **Risk-first execution:** every order passes explicit, versioned controls.
4. **Broker independence:** normalized internal models with replaceable adapters.
5. **Local-first privacy:** strategies and broker credentials remain under customer control wherever possible.
6. **Operational clarity:** every rejection, fill, position and P&L change is explainable.
7. **Professional UX:** keyboard-driven, high-information-density workflows without retail-style visual clutter.

4. Market rollout

Initial engineering integration

Use **Interactive Brokers paper trading** as the first broker implementation.

Its current API documentation covers market data, account information, order placement, order monitoring, positions, portfolio values and multiple order types. Its TWS API is a socket-based interface available through languages including Python, Java, C++ and C#.

However, broker market data is not equivalent to owning unrestricted data. Interactive Brokers notes that API users generally require relevant market-data subscriptions.

Recommended sequence

Release 1

- Equities and ETFs.
- One broker.
- Paper and live trading.
- Bar and quote data.
- Single-account operation.
- Market, limit, stop and bracket orders.
- End-of-day and intraday strategies.

Release 2

- Listed options.
- Multi-leg orders.
- Greeks and volatility surfaces.
- Portfolio-level options risk.
- Futures through the existing broker adapter.

Release 3

- Second broker.
- Multiple accounts.
- Allocation and rebalancing.
- FIX adapter for professional clients.
- Team permissions and approval workflows.

Release 4

- India-specific broker integration.
- Regulatory workflow appropriate to SEBI and the selected broker.
- India-specific contract, expiry, margin and market-session logic.

SEBI issued a retail algorithmic-trading framework in February 2025 and later extended its implementation timeline. This means an Indian commercial launch must be designed with the broker and securities counsel rather than treating broker APIs as unrestricted execution endpoints.

5. Complete product capability map

P0: Mandatory commercial core

5.1 Instrument and reference-data service

Maintain a canonical instrument model containing:

- Internal instrument ID.
- Symbol and exchange symbol.
- Asset class.
- Venue.
- Currency.
- Tick size.
- Lot size.
- Multiplier.
- Trading calendar.
- Settlement rules.
- Expiry.
- Strike and option right.
- Corporate-action history.
- Broker-specific identifiers.

Never use ticker symbols as permanent identifiers.

5.2 Market-data engine

Responsibilities:

- Normalize quotes, trades and bars.
- Record source timestamps and local receive timestamps.
- Detect stale, delayed and missing data.
- Construct bars deterministically.
- Handle exchange sessions and holidays.
- Store raw and normalized events.
- Replay historical sessions using the same event interface as live data.
- Detect sequence gaps where the source provides sequencing.

5.3 Research and strategy SDK

Provide a Python SDK for:

- Subscribing to market events.
- Requesting historical data.
- Defining indicators and features.
- Generating order intents.
- Reading portfolio state.

- Receiving fills and risk decisions.
- Saving strategy state.
- Emitting metrics and structured logs.

A strategy must never directly call a broker adapter.

It submits an `OrderIntent` to the platform. The risk and order-management layers decide whether and how the intent becomes an executable broker order.

5.4 Event-driven backtester

The backtester must support:

- The same strategy API used in production.
- Point-in-time data.
- Corporate actions.
- Delistings.
- Survivorship-bias controls.
- Transaction fees.
- Exchange and regulatory charges.
- Bid/ask spreads.
- Configurable slippage.
- Partial fills.
- Order latency.
- Session boundaries.
- Market halts.
- Borrow costs and short-sale constraints.
- Deterministic random seeds.
- Portfolio-level capital constraints.

A backtest without these controls is a chart generator, not a professional research system.

5.5 Order-management system

The OMS owns:

- Client order IDs.
- Broker order IDs.
- Order lifecycle.
- Parent-child relationships.
- Replace and cancel operations.
- Partial fills.
- Rejections.
- Time-in-force handling.
- Broker reconnection.
- Duplicate-event handling.
- Recovery following restarts.
- End-of-day reconciliation.

The state machine should include explicit states such as:

```
CREATED
PENDING_RISK
RISK_REJECTED
APPROVED
PENDING_SUBMIT
SUBMITTED
ACKNOWLEDGED
PARTIALLY_FILLED
FILLED
PENDING_CANCEL
CANCELLED
REJECTED
EXPIRED
UNKNOWN
```

`UNKNOWN` is essential. A network interruption must never cause the platform to assume that an order failed.

5.6 Execution-management system

Initial execution methods:

- Immediate market or limit order.
- Passive limit order.
- Bracket order.
- Time-weighted execution.
- Participation-limited execution.
- Cancel-and-replace policies.
- Price-chasing with strict boundaries.
- Basket submission with portfolio checks.

Later:

- VWAP.
- POV.
- Arrival-price algorithms.
- Multi-venue routing.
- Options combination execution.

5.7 Pre-trade risk engine

Every order must pass:

- Instrument permission.

- Market-session validation.
- Trading-halt check.
- Market-data freshness check.
- Price collar.
- Maximum quantity.
- Maximum notional.
- Maximum open-order count.
- Maximum orders per second.
- Buying-power or margin reserve.
- Symbol exposure.
- Sector exposure.
- Asset-class exposure.
- Currency exposure.
- Gross and net portfolio exposure.
- Strategy allocation.
- Daily realized-loss limit.
- Daily total-loss limit.
- Maximum drawdown state.
- Duplicate-order detection.
- Self-trade prevention.
- Concentration limits.
- Restricted-list validation.
- Global, account, strategy and symbol kill switches.

FINRA's algorithmic-trading guidance specifically emphasizes controlled software changes, archived versions, pilot deployments, segregated test environments, real-time monitoring, risk-control parameter review and mechanisms to disable algorithms quickly.

In the United States, Rule 15c3-5 requires market-access broker-dealers to maintain risk-management and supervisory controls under their direct and exclusive control. Your product's controls should therefore complement—not attempt to replace—the broker's controls.

5.8 Portfolio and accounting engine

Track independently from the broker:

- Cash by currency.
- Positions.
- Average cost.
- Realized P&L.
- Unrealized P&L.
- Fees and commissions.
- Dividends.
- Corporate actions.
- FX translation.
- Buying power.
- Margin estimates.
- Strategy attribution.

- Account attribution.

Then reconcile the internal ledger against the broker.

Differences must generate incidents, not silent corrections.

5.9 Risk cockpit

The primary risk screen should display:

- Current and projected exposure.
- Gross and net leverage.
- Daily P&L.
- Drawdown.
- Risk-limit utilization.
- Open orders.
- Stale feeds.
- Broker connection health.
- Strategy heartbeat.
- Concentration.
- Options Greeks.
- Margin utilization.
- Active restrictions.
- Kill-switch status.

5.10 Audit and replay

Every significant operation should create an immutable event:

```
event_id
event_type
schema_version
event_time
receive_time
account_id
strategy_id
instrument_id
correlation_id
causation_id
actor
source
payload
software_version
configuration_version
```

Given an incident ID, the system should reconstruct:

- What market data was available.
- Which strategy version ran.
- Which configuration applied.
- Why the order was generated.
- Which risk checks ran.
- Why an order was accepted or rejected.
- What was sent to the broker.
- What the broker returned.
- How positions and P&L changed.

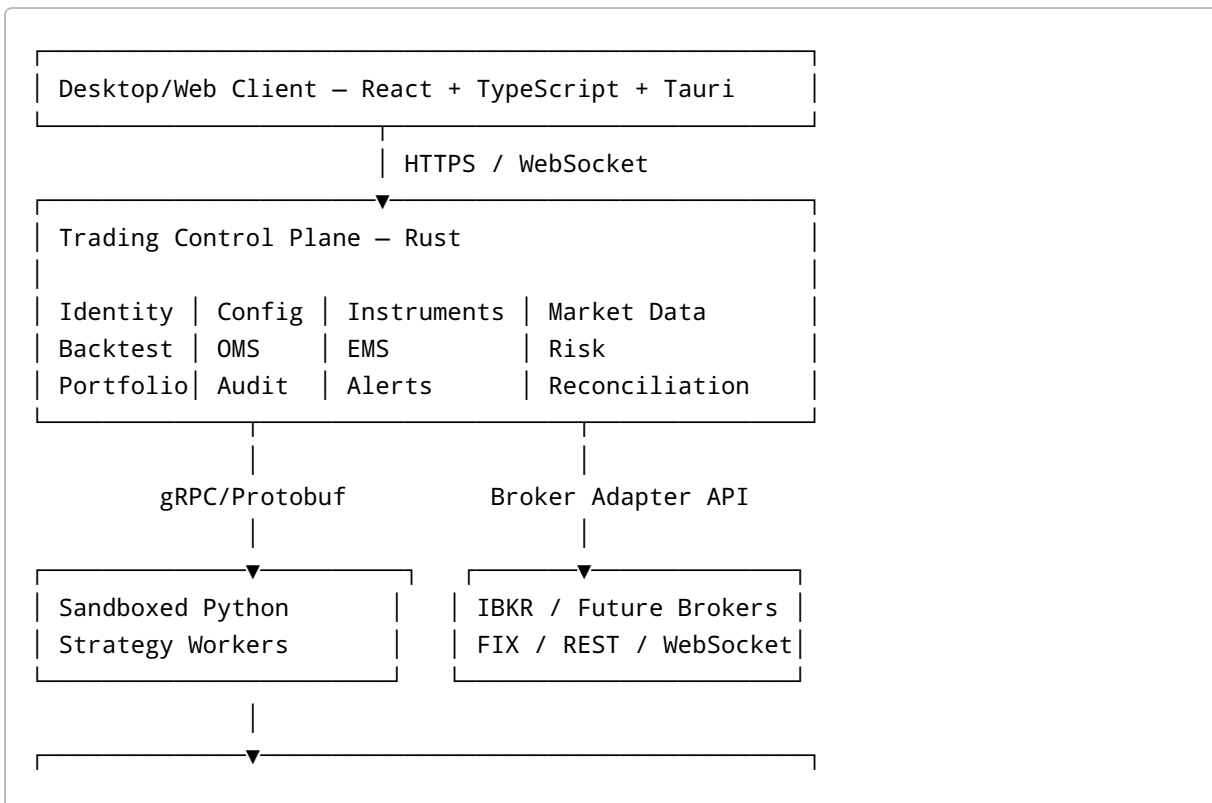
That capability is a major component of the product's defensibility.

6. Recommended technical architecture

Architecture principle

Start with a **modular monolith plus isolated strategy workers**.

Do not begin with microservices.



Technology choices

Core trading engine

Rust

Use it for:

- Domain models.
- OMS.
- Risk engine.
- Broker adapters.
- Market-data normalization.
- Portfolio state.
- Replay.
- Control-plane APIs.

Reasons:

- Strong type system.
- Memory safety.
- Predictable performance.
- Excellent concurrency model.
- Suitable for long-running stateful systems.

Strategy and research environment

Python

Use it for:

- Strategy code.
- Statistical research.
- Machine learning.
- Notebook integration.
- Optimization.
- Feature engineering.

Python runs outside the risk-critical core.

Frontend

React + TypeScript

Use:

- A browser version for remote access.
- Tauri packaging for a professional desktop application.
- WebSocket streaming for live state.
- A high-quality financial charting component rather than writing a renderer from zero.

Primary database

PostgreSQL

Use it for:

- Users.
- Accounts.
- Configurations.
- Orders.
- Executions.
- Positions.
- Risk limits.
- Audit indexes.
- Strategy metadata.
- Billing metadata.

Analytical storage

Parquet plus DuckDB

Use it for:

- Historical bars.
- Tick data.
- Backtest datasets.
- Feature matrices.
- Local analytical queries.
- Portable customer exports.

Object storage

Use an S3-compatible service for:

- Raw market-data files.
- Backtest artifacts.
- Reports.
- Logs.
- Strategy bundles.
- Database backups.
- Signed release artifacts.

Internal communication

Use:

- Protobuf schemas.
- gRPC for strategy-worker communication.
- WebSockets for frontend streaming.
- REST for administrative APIs.

Do not add Kafka or Kubernetes initially. Introduce durable distributed messaging only when the modular monolith can no longer satisfy measured requirements.

7. Repository structure

```
/apps
  /desktop
  /web
  /cli

/core
  /domain
  /instrument
  /market-data
  /replay
  /backtest
  /oms
  /execution
  /risk
  /portfolio
  /reconciliation
  /audit
  /alerts

/adapters
  /brokers
    /ibkr
  /market-data
  /notifications

/python
  /strategy-sdk
  /research
  /examples

/contracts
```

```
/protobuf
/json-schema

/infra
  /terraform
  /containers
  /monitoring

/docs
  /architecture
  /adr
  /operations
  /security
  /compliance
  /user-guide

/tests
  /simulation
  /replay
  /integration
  /fault-injection
```

Every major architectural decision should have an Architecture Decision Record.

8. UX and product design

Core design principles

Risk before profit

Display exposure, loss limits and system health more prominently than projected returns.

SEBI reported that 93% of individual equity F&O traders in its FY22–FY24 study incurred losses. This reinforces the need to position the product around disciplined process and risk management rather than promised profitability.

Separate intention from execution

The interface must clearly distinguish:

- Strategy signal.
- Order intent.
- Risk approval.
- Broker submission.

- Broker acknowledgement.
- Execution.
- Portfolio effect.

Make live mode unmistakable

Paper and live trading should never be visually confusable.

Live mode should require:

- Explicit account selection.
- Visible environment indicator.
- Persistent risk-limit display.
- Confirmation for dangerous actions.
- Optional hardware or passkey approval for global-limit changes.

Explain every rejection

Never display merely:

Order rejected

Display:

Rejected by PRICE_COLLAR

Submitted limit: 193.40
 Reference ask: 191.10
 Maximum deviation: 0.75%
 Actual deviation: 1.20%
 Risk policy: US_EQUITY_DEFAULT_V7

Primary screens

1. **Command Center** — system, broker, strategies and risk status.
2. **Research Lab** — notebooks, datasets and experiments.
3. **Strategy Studio** — configuration, versioning and deployment.
4. **Backtest Explorer** — results, trades, regimes and sensitivity.
5. **Execution Blotter** — intents, orders, fills and rejections.
6. **Risk Cockpit** — exposure, limits, drawdown and kill switches.
7. **Portfolio** — positions, attribution and scenarios.
8. **Replay and Incidents** — reconstruct production behavior.
9. **Journal** — decisions, annotations and review.
10. **Administration** — credentials, accounts, permissions and audit.

9. Reliability and quality requirements

Non-negotiable invariants

- Every accepted order intent has a traceable terminal or explicitly unresolved state.
- Duplicate broker messages do not produce duplicate fills.
- Restarting the application cannot silently lose working orders.
- Internal positions are continuously reconciled against broker positions.
- A strategy cannot bypass risk checks.
- Market-data staleness stops affected strategies.
- Configuration changes are versioned and attributable.
- Live software versions are reproducible from source control.
- All timestamps are stored in UTC alongside exchange-local context.
- Monetary arithmetic uses decimal or fixed-point representations, never binary floating-point for accounting.
- Every kill switch works without the strategy process.
- Broker disconnection defaults to a documented safe state.

Testing strategy

Unit tests

For pure calculations and domain rules.

Property-based tests

Especially for:

- OMS transitions.
- Fill aggregation.
- Position accounting.
- Currency conversions.
- Risk-limit boundaries.
- Serialization.
- Idempotency.

Model-based state-machine tests

Generate order events in unusual sequences:

- Fill before acknowledgement.
- Cancel while partially filled.
- Duplicate execution.
- Disconnect during replace.
- Rejection following apparent submission.
- Late broker status.

- Session restart with working orders.

Deterministic replay tests

The same input events and configuration must generate identical output events.

Simulation tests

Create synthetic exchanges and brokers with:

- Delays.
- Packet loss.
- Duplicate messages.
- Reordering.
- Partial fills.
- Trading halts.
- Fast markets.
- Broken clocks.
- Rate limits.
- Authentication expiry.

Shadow mode

Before allowing a strategy to trade:

- Run it against live data.
- Generate intended orders.
- Do not transmit them.
- Compare expected fills and risk behavior.
- Review for several market regimes.

Capital canary

Live deployment sequence:

```
Replay
→ Historical simulation
→ Paper trading
→ Shadow mode
→ Minimum-size live trading
→ Restricted capital
→ Normal allocation
```

Nasdaq itself provides test facilities so firms can validate automated systems before production deployment, reflecting how seriously production testing is treated in professional market infrastructure.

Target service objectives

Treat these as engineering targets:

- 100% audit coverage for trading actions.
 - Zero plaintext broker credentials.
 - Risk-check latency below 5 ms at the local core's p99.
 - No loss of persisted order or execution events after acknowledgement.
 - Production availability of at least 99.9% before enterprise sales.
 - Recovery within 15 minutes from ordinary control-plane failure.
 - Automated reconciliation after every reconnect.
 - Daily restore verification for critical backups.
-

10. Security architecture

Use NIST's Secure Software Development Framework as the development-process baseline and OWASP ASVS Level 2 as the initial application-security verification target. NIST describes SSDF as a set of secure-development practices integrated into the software lifecycle, while OWASP ASVS provides concrete technical security verification requirements.

Required controls

- Passkeys or MFA.
- Short-lived sessions.
- Role-based access control.
- Separate privileges for trading and administration.
- Operating-system keychain for local secrets.
- Managed secret storage in hosted deployments.
- Envelope encryption for stored credentials.
- TLS for every network connection.
- Database encryption at rest.
- Immutable security audit trail.
- Signed application releases.
- Software bill of materials.
- Dependency pinning.
- Automated dependency scanning.
- Static application-security testing.
- Dynamic application-security testing.
- Secret scanning.
- Protected branches.
- Mandatory CI checks.
- Reproducible production builds.
- Rate limits.
- Request idempotency.

- Secure broker-token rotation.
- Session revocation.
- External penetration test before broad commercial release.

Never expose broker credentials to strategy code.

11. Market-data strategy

Market data is both a technical and licensing problem.

Do not include unrestricted exchange data inside the subscription by default. Nasdaq requires distributors of controlled real-time data to administer applicable subscriber agreements, demonstrating that commercial redistribution has contractual obligations.

Use one of these models:

1. **Bring your own market-data entitlement.**
2. **Broker-provided data attached to the customer's account.**
3. **Customer contracts directly with a data vendor.**
4. **Your company becomes an authorized distributor later.**

For the first commercial release, use the first three models.

Avoid becoming a data redistributor until revenue can support legal review, reporting and exchange fees.

12. Compliance posture

Initial operating model

The safest initial structure is:

- Users maintain their own brokerage accounts.
- Users authorize the product to connect.
- The platform does not custody funds.
- The platform does not promise returns.
- The platform does not rank public strategies by expected profitability.
- The platform does not personalize securities recommendations.
- Users define and approve their strategies and limits.
- The product maintains explicit records of user actions and automation.

SEBI has specifically addressed performance and return claims made by unregulated algorithmic-trading platforms, so marketing must avoid guaranteed or exaggerated results.

Before public live trading, obtain advice covering:

- Investment-adviser classification.
- Research-analyst classification.
- Broker/API terms.
- Automated-trading requirements.
- Data licensing.
- Privacy.
- Cybersecurity obligations.
- Record retention.
- Customer disclosures.
- Tax treatment.
- Cross-border service restrictions.

Compliance is part of product architecture, not a final launch checklist.

13. Twenty-four-month development roadmap

Months 0–2: Product and domain foundation

Deliver:

- Product charter.
- Ideal-customer definition.
- Non-goals.
- Domain glossary.
- Canonical schemas.
- Threat model.
- Initial compliance memo.
- Repository and CI.
- PostgreSQL schema.
- Event envelope.
- Instrument model.
- Market calendar.
- Architecture Decision Records.
- Basic desktop shell.

Exit criterion:

Historical market events can be ingested, stored and replayed deterministically.

Months 3–5: Data and backtesting

Deliver:

- Data importer.
- Bar builder.
- Corporate-action adjustment.
- Python strategy SDK.
- Event-driven backtester.
- Fees and slippage.
- Portfolio accounting.
- Backtest reports.
- Experiment metadata.
- Reproducible strategy bundles.

Exit criterion:

A strategy can run repeatedly with identical results from a versioned dataset and configuration.

Months 6–8: OMS, risk and paper execution

Deliver:

- Complete order state machine.
- Risk engine.
- IBKR paper adapter.
- Order blotter.
- Position reconciliation.
- Kill switches.
- Broker reconnect logic.
- Fault-injection test broker.
- Paper-trading dashboard.

Exit criterion:

The platform can operate continuously in paper mode for 30 trading days without unexplained position or order discrepancies.

Months 9–11: Controlled live trading

Deliver:

- Live account support.
- Credential security.
- Deployment approvals.

- Shadow mode.
- Minimum-capital live mode.
- Incident management.
- Daily reconciliation.
- Disaster-recovery procedure.
- Automated backup restoration.
- Production monitoring.

Exit criterion:

Small-capital live trading completes for 60 trading days with complete auditability and no unresolved accounting discrepancies.

Months 12–14: Professional workflow

Deliver:

- Risk cockpit.
- Portfolio attribution.
- Trading journal.
- Alerts.
- Strategy scheduling.
- Parameter management.
- Configuration comparison.
- Replay UI.
- Incident timeline.
- Exportable reports.
- High-quality documentation.

Exit criterion:

Five design partners can use the platform without direct developer assistance for normal workflows.

Months 15–17: Options and portfolio risk

Deliver:

- Options instrument model.
- Option chains.
- Greeks.
- Volatility surfaces.
- Multi-leg orders.
- Scenario analysis.
- Portfolio Greek limits.
- Expiry handling.
- Assignment and exercise workflows.

Exit criterion:

Options positions reconcile and remain reproducible across backtest, paper and live environments.

Months 18–20: Commercial hardening

Deliver:

- Billing.
- Licensing.
- Account provisioning.
- Support tooling.
- Privacy controls.
- Data-retention controls.
- External penetration test.
- Security remediation.
- Operational runbooks.
- Customer onboarding.
- Signed releases.
- Self-hosted deployment option.

Exit criterion:

Ten paying professional customers or three paying organizations.

Months 21–24: Expansion

Choose only one:

- Second broker.
- India-specific regulated integration.
- FIX connectivity.
- Multi-account allocation.
- Team collaboration and approvals.

Do not attempt all five simultaneously.

Exit criterion:

The selected expansion produces measurable paid demand.

14. Solo-founder operating system

Time allocation

A sustainable allocation is:

- 50% core product engineering.
- 20% automated testing and reliability.
- 10% security and operations.
- 10% documentation.
- 10% customer research and support.

Do not reduce testing to create more visible features.

Weekly cadence

Monday

- Review incidents.
- Review risk.
- Select one weekly outcome.
- Write or update the design document.

Tuesday–Thursday

- Implementation.
- Tests.
- Benchmarks.
- Documentation alongside code.

Friday

- Full replay suite.
- Fault-injection suite.
- Security scans.
- Release candidate.
- User demonstration.
- Retrospective and decision log.

Limit work in progress to one major feature.

Design-partner cadence

Beginning in month four:

- Interview two professional traders every month.

- Observe their current workflow.
 - Record repetitive manual operations.
 - Identify where errors occur.
 - Build only workflows for which users demonstrate pain.
 - Charge early enough to distinguish interest from willingness to pay.
-

15. Commercial model

Recommended pricing structure

Professional

- \$199–\$399 per month.
- One user.
- One live account.
- Research, backtesting and execution.
- Standard support.

Quant

- \$499–\$999 per month.
- Advanced data workflows.
- Multiple strategies.
- Options analytics.
- Higher limits.
- Priority support.

Team

- \$12,000–\$30,000 per year.
- Multiple users.
- Permissions.
- Approval workflows.
- Central risk.
- Audit exports.
- Deployment support.

Private deployment

- \$50,000 or more per year.
- Customer-controlled cloud or infrastructure.
- Enterprise authentication.
- Custom retention.
- Premium support.
- Contracted service levels.

These are target positions, not initial launch prices. Validate them with design partners.

Revenue paths supporting a \$1 million-class outcome

Illustrative scenarios:

50 professional users × \$499/month
≈ \$299,000 annual recurring revenue

12 small firms × \$25,000/year
= \$300,000 annual recurring revenue

5 private deployments × \$60,000/year
= \$300,000 annual recurring revenue

A buyer will examine more than revenue. The product should also have:

- Low customer concentration.
- Strong retention.
- Documented proprietary IP.
- Clean ownership of all source code.
- Assignable customer and vendor contracts.
- Reproducible builds.
- Security assessments.
- Operational documentation.
- Reliable recurring revenue.
- Limited founder-dependent support.
- Demonstrated broker and market extensibility.

Do not treat “\$1 million worth” as a feature requirement. Treat it as a business-quality scorecard.

16. Product-value scorecard

The platform is approaching the intended quality level when it has:

Technical

- Deterministic research-to-live replay.
- A production-quality OMS.
- Layered risk controls.
- Reliable broker reconciliation.
- Complete auditability.
- Secure credential management.

- Documented disaster recovery.
- External penetration-test results.
- At least one year of live operational history.
- A second integration proving the adapter architecture.

Product

- Five-minute onboarding into paper mode.
- Clear live/paper separation.
- Every order decision explainable.
- Professional keyboard-driven UX.
- Excellent documentation.
- Self-service diagnostics.
- Low support burden.

Commercial

- At least \$250,000 in annual recurring revenue, or a credible path toward it.
 - Ten or more paying professional organizations, or a larger diversified professional-user base.
 - Strong annual retention.
 - Multiple customer references.
 - No single customer controlling the roadmap.
 - Clean contracts and intellectual-property ownership.
-

17. First 30 days

Week 1: Product contract

Produce:

- One-page product charter.
- Customer definition.
- Non-goals.
- Complete workflow diagram.
- Risk philosophy.
- Initial jurisdiction decision.
- Legal-question list.
- Twenty customer-interview questions.

Week 2: Domain skeleton

Implement:

- Rust workspace.

- Python strategy SDK repository.
- TypeScript desktop shell.
- CI pipeline.
- Event envelope.
- Instrument model.
- Order intent model.
- Risk-decision model.
- Execution model.
- PostgreSQL migrations.

Week 3: Recorder and replay

Implement:

- Historical data importer.
- Normalized bar events.
- Event storage.
- Replay clock.
- Strategy subscription.
- Structured logging.
- Deterministic replay test.

Week 4: First vertical slice

Complete this exact flow:

```

Historical event
→ Python strategy
→ Order intent
→ Risk decision
→ Simulated order
→ Simulated fill
→ Position update
→ P&L update
→ Audit event
→ Desktop display

```

Do not build indicators, options analytics, AI or multiple brokers until this vertical slice is correct.

18. Final engineering doctrine

1. **Correctness before speed.**
2. **Risk before convenience.**

3. **One excellent workflow before ten partial workflows.**
4. **Replay before live trading.**
5. **Audit every decision.**
6. **Treat disconnections and duplicate events as normal conditions.**
7. **Make invalid states difficult to represent.**
8. **Never allow strategy code to bypass the trading kernel.**
9. **Use managed infrastructure until scale proves otherwise.**
10. **Build proprietary domain intelligence, not commodity infrastructure.**
11. **Sell reliability and control—not promised returns.**
12. **Expand markets through adapters only after the core is stable.**

The optimal ambition is not:

“One person building Jane Street.”

It is:

“One person building the most trustworthy, integrated and technically rigorous trading operating system available to independent professionals.”

That product is difficult but achievable, defensible and potentially valuable enough to become a seven-figure software business.